

Neural Network

Prof. Giuseppe Serra

Università degli Studi di Udine

Should I watch the movie "Gravity"?

I can ask my close friends Mary and John



Both of them Give the movie a rating of 3 in the scale between 1 to 5

It is difficult for me to decide.....

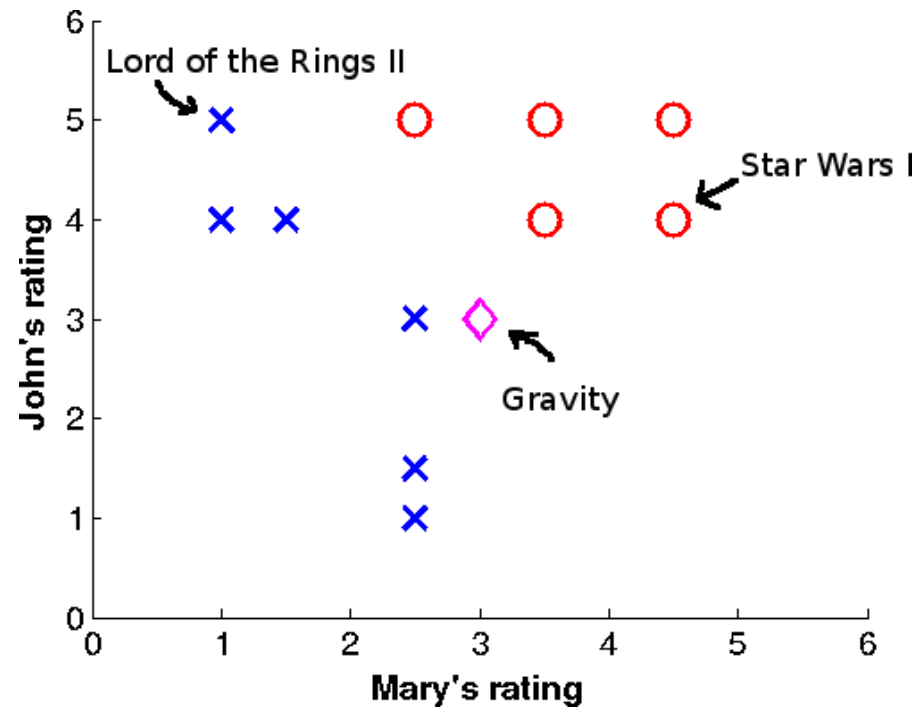


Maybe I can use their ratings of past movies?

Movie name	Mary's rating	John's rating	I like?
Lord of the Rings II	1	5	No
...	...	...	...
Star Wars I	4.5	4	Yes
Gravity	3	3	?

Let's visualize the data to see if there is any trend...

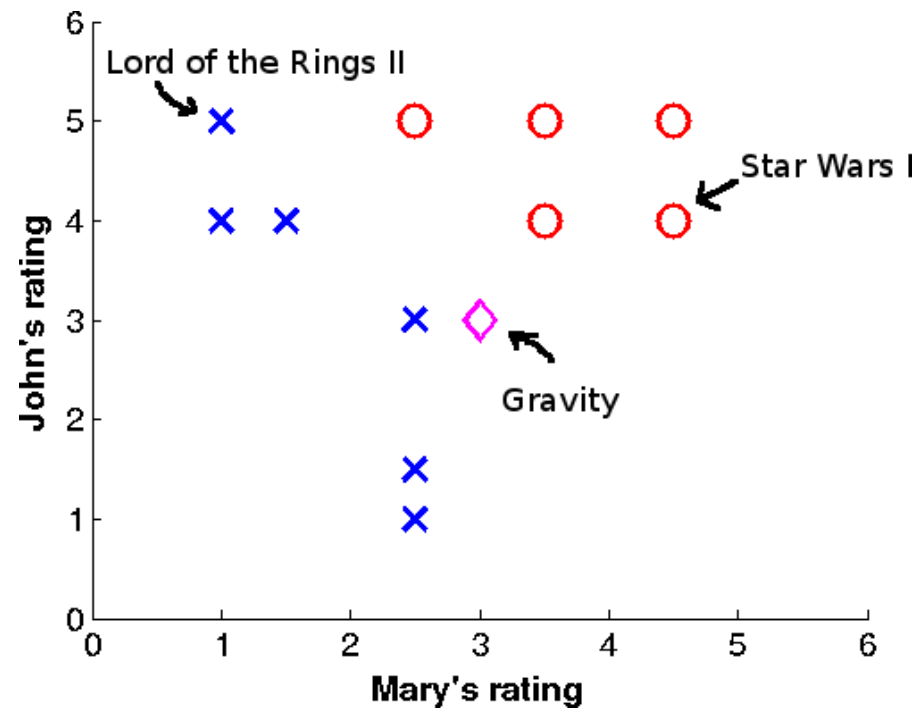
Maybe I can use their ratings of past movies?



○ corresponds to "I like the movie"

× corresponds to "I dislike the movie"

I can use a Logistic Regression to take a decision!



○ corresponds to "I like the movie"

× corresponds to "I dislike the movie"

Logistic Regression (recall)

Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)}), \}$

The goal is to find θ so
$$\begin{cases} h_{\theta}(x) \geq 0.5 & \text{if } y = 1 \\ h_{\theta}(x) < 0.5 & \text{if } y = 0 \end{cases}$$

$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1+e^{-\theta^T x}}$, where $g(z)$ is the Sigmoid function,

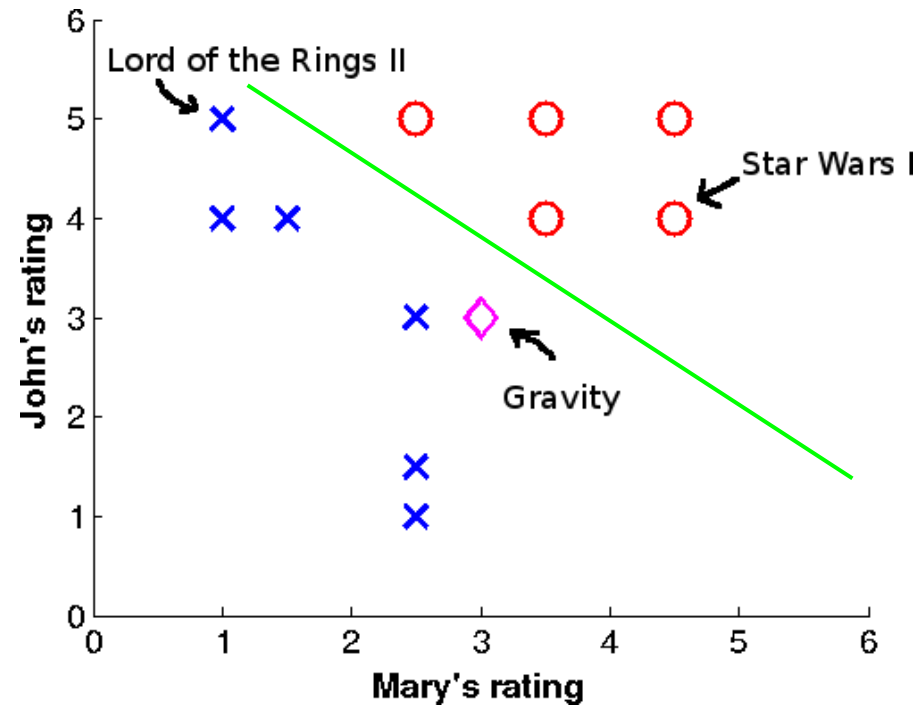
$$\theta = \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_n \end{bmatrix}; \quad x = \begin{bmatrix} 1 \\ \vdots \\ x_n \end{bmatrix}$$

Cost function:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x), y)$$

where $\text{Cost}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$

I can use a Logistic Regression to take a decision



- ○ corresponds to "I like the movie"
- X corresponds to "I dislike the movie"

Based on this decision boundary, "Gravity" is on the negative side

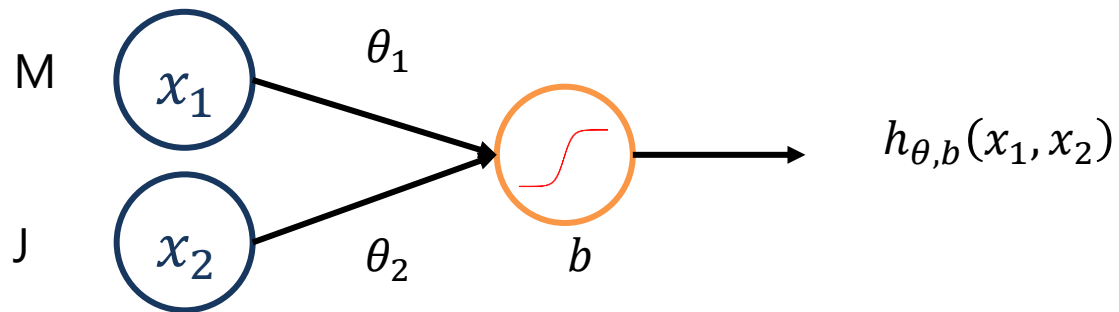
Logistic Regression (recall) – changing notation

$$\text{Let's } \theta = \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}; \quad x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}; \quad b = \theta_0$$

The hypothesis of the Logistic Regression is:

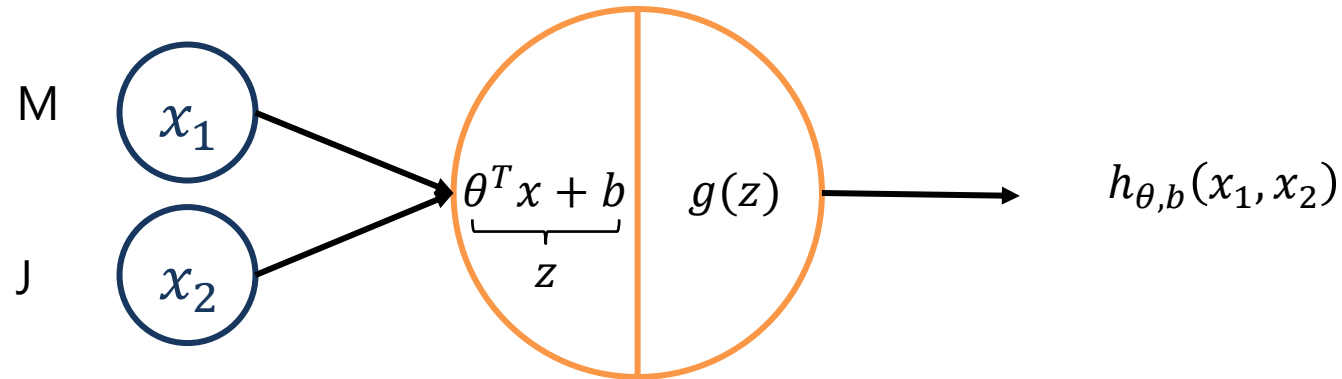
$$h_{\theta,b}(x) = g(\theta^T x + b) = \frac{1}{1 + e^{-(\theta^T x + b)}}, \text{ where } g(z) \text{ is the Sigmoid function,}$$

Logistic Regression Representation - I



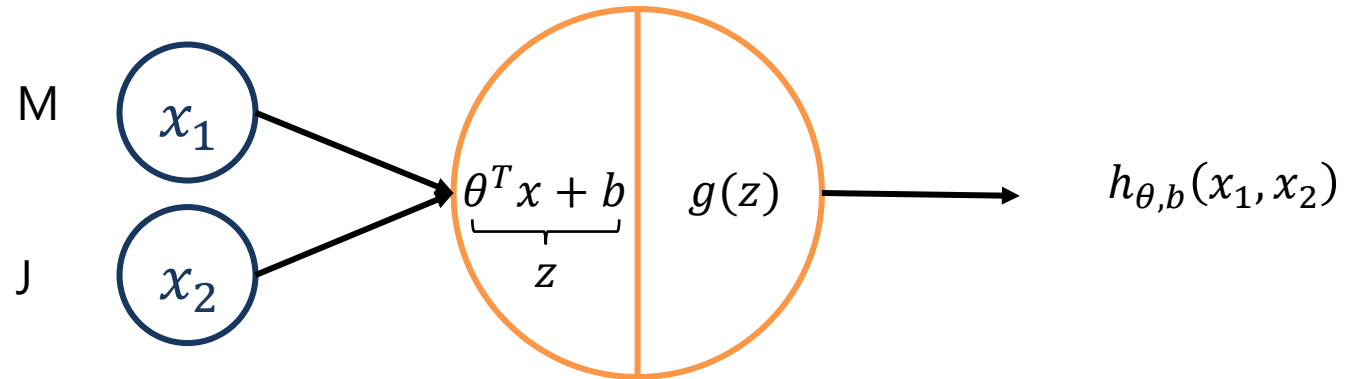
This network means that to compute the value of the decision function, we need to multiply Mary's rating with θ_1 , John's rating with θ_2 , then add two values and b , then apply the sigmoid function

Logistic Regression Representation - II

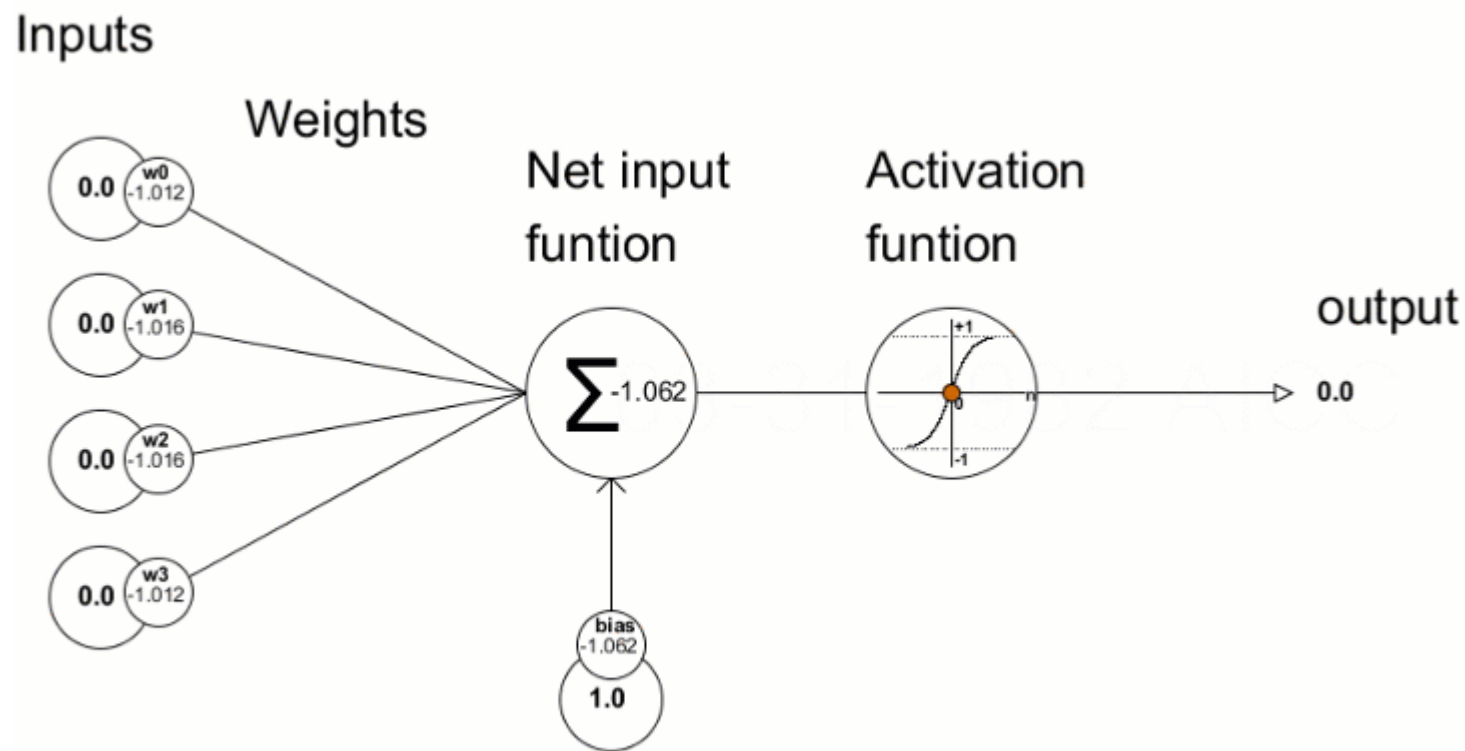


This network means that to compute the value of the decision function, we need to multiply Mary's rating with θ_1 , John's rating with θ_2 , then add two values and b , then apply the sigmoid function

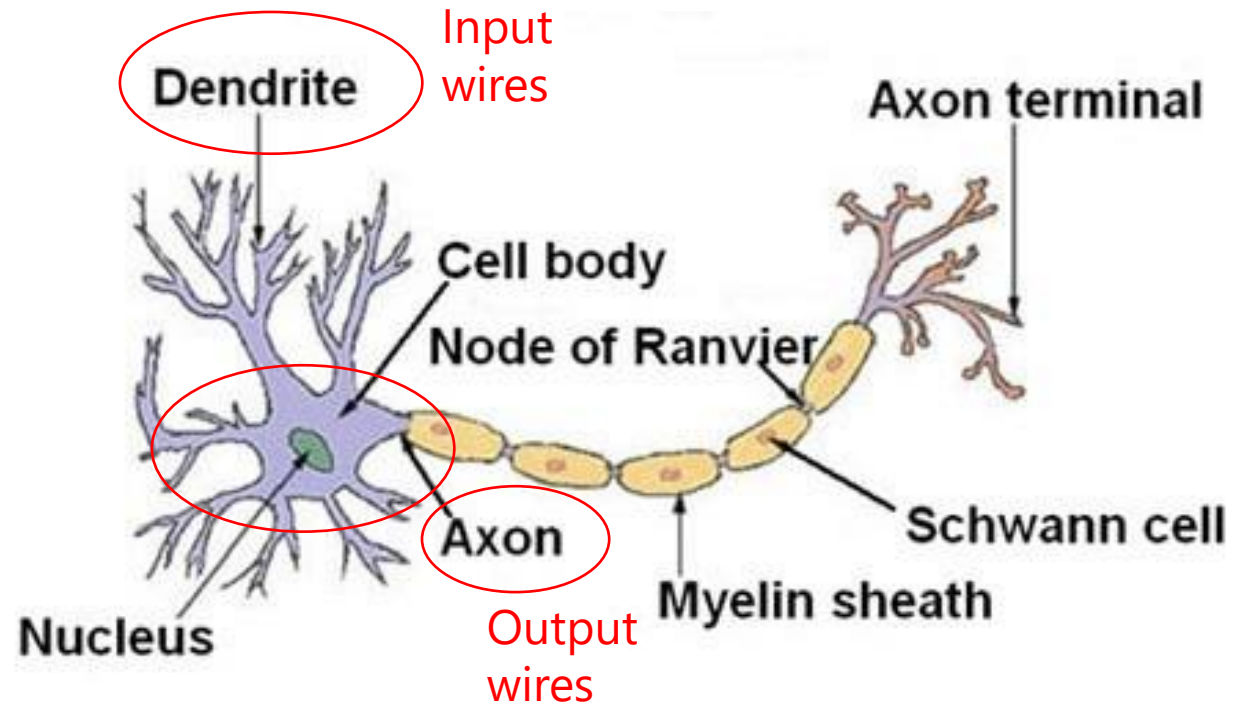
Neural Network Representation



Neural Networks

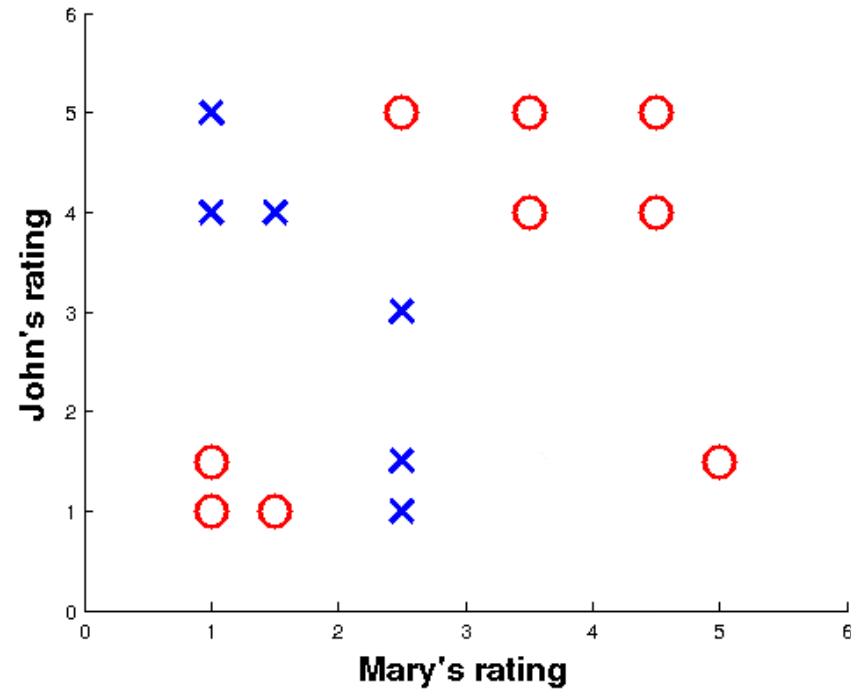


Neural in the Brain



Complex Problem

Now consider Susan that has different movies tastes (Complex Problem)



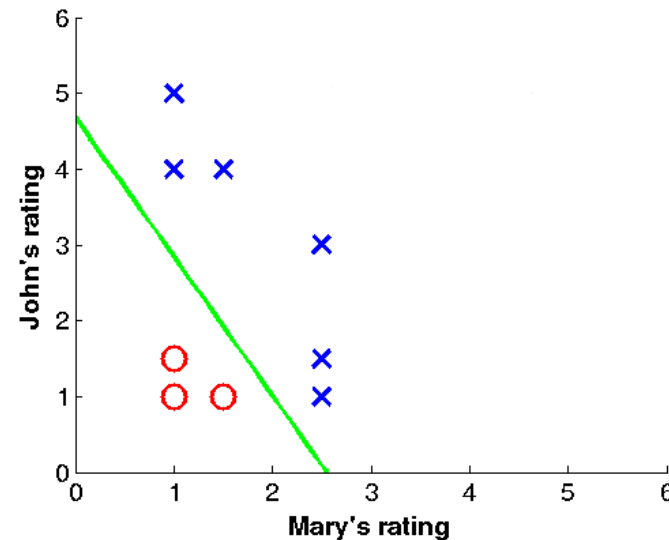
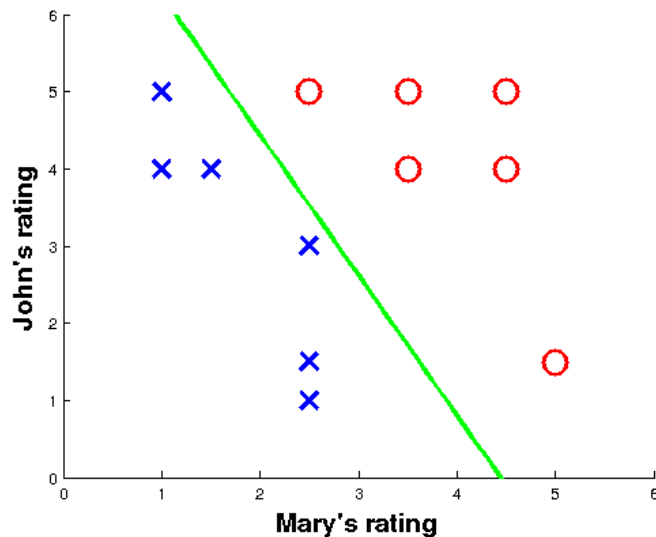
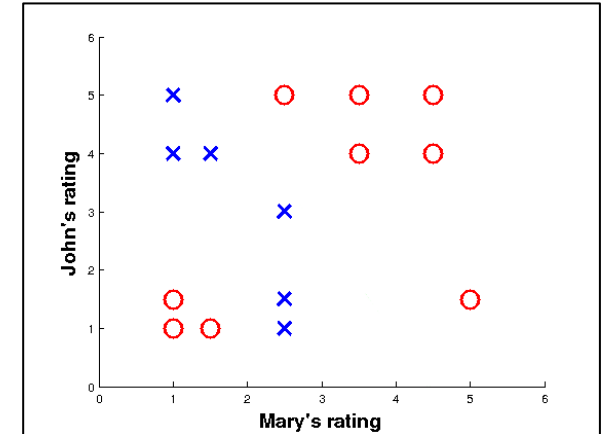
○ corresponds to "I like the movie"

× corresponds to "I dislike the movie"

Complex Problem

Decomposition of a complex problem in smaller problems

If we throw away the "weird" samples...



Decision function of decision functions

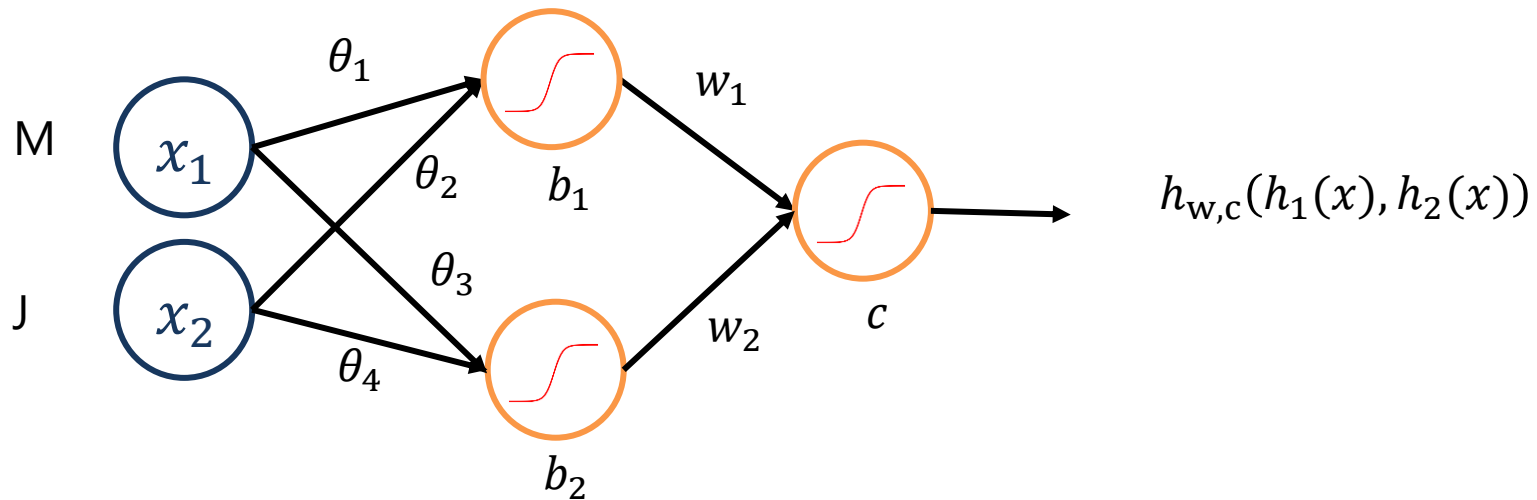
Movie name	Output by decision function h_1	Output by decision function h_2	Susan likes?
Lord of the Rings II	$h_1(x^{(1)})$	$h_2(x^{(2)})$	No
...	...	...	...
Star Wars I	$h_1(x^{(n)})$	$h_2(x^{(n)})$	Yes
Gravity	$h_1(x^{(n+1)})$	$h_2(x^{(n+1)})$	?

Using a Logistic Regression, the goal is to find a new parameter set θ and b of $h_{w,b}(h_1(x), h_2(x))$ to predict Susan's movie tastes.

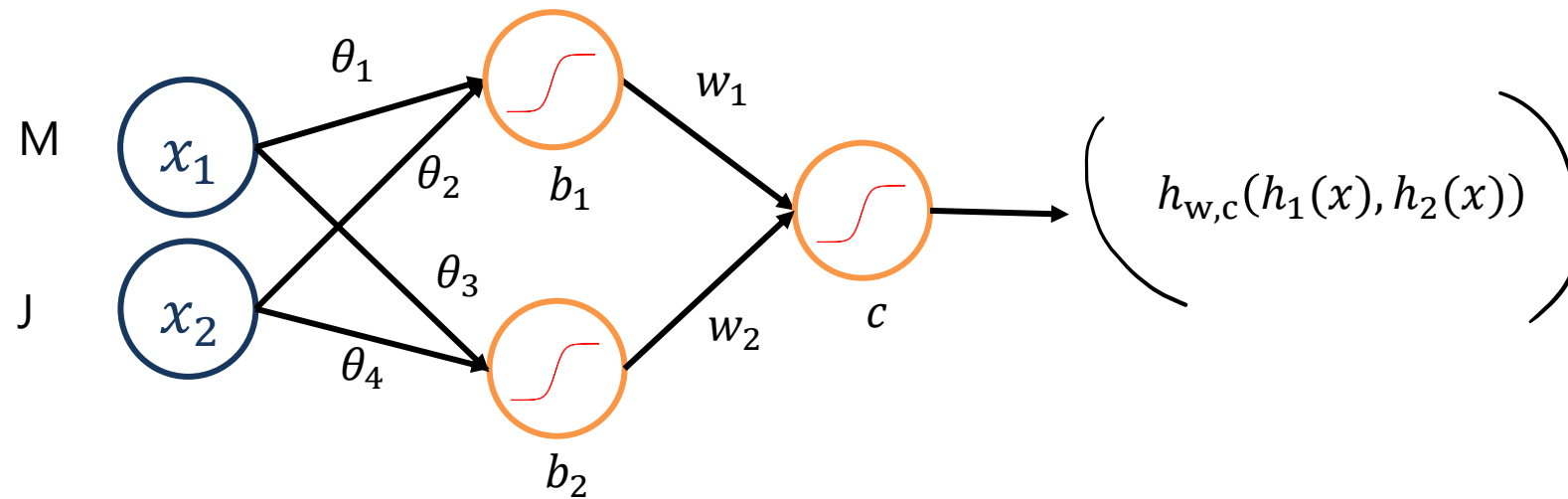
Decision Function

We can find the decision function for Susan by following two steps:

1. Partition the data into two sets (as seen before). Each set can be simply classified by a Logistic Regression.
2. Use the newly-found decision functions (using Logistic Regression) and compute the decision values for each example. Then treat these values as input to another Logistic Regression.



Neural Network with One Hidden Layer



Input Layer

Hidden Layer

Output Layer

Complex Problem

I cheated a Little bit

I cheated a little bit when I divided the dataset into two set ...

I looked at the data and decided that the two set should be portioned.

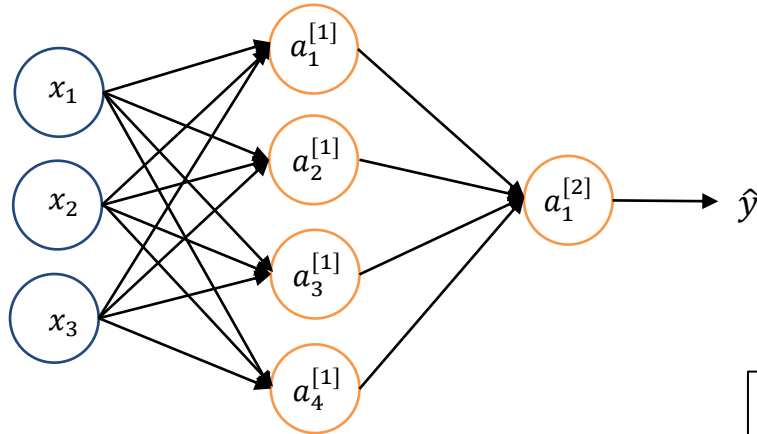
Can we do it automatically?

Yes!

You can find all these parameters $w_1, w_2, c, \theta_1, \theta_2, \theta_3, \theta_4, b_1, b_2$ using a famous algorithm commonly known as the backpropagation algorithm (see next slides)

Try by yourself: <https://phiresky.github.io/neural-network-demo/>

Neural Network



$a^{[j]}$ = activation of vector for layer j
 $W^{[j]}$ = matrix of weights controlling function mapping from layer $j - 1$ to layer j
 $b^{[j]}$ = bias

Given in input X:

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

(4,1) (4,3) (3,1) (4,1) - Dimensionality

$$a^{[1]} = g(z^{[1]})$$

(4,1) (4,1) - Dimensionality

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

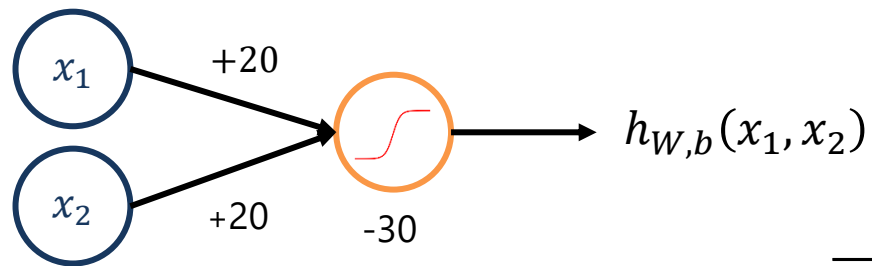
(1,1) (1,4) (4,1) (1,1) - Dimensionality

$$a^{[2]} = g(z^{[2]})$$

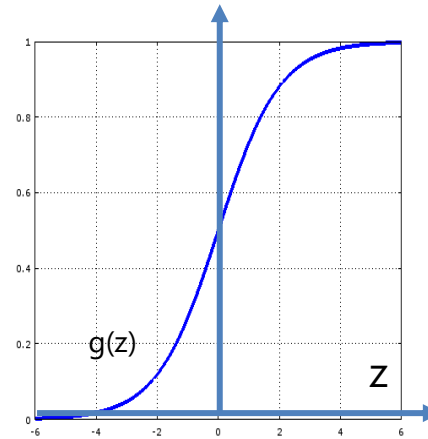
(1,1) (1,1) - Dimensionality

Example: AND

$$x_1, x_2 \in \{0, 1\}$$
$$y = x_1 \text{ AND } x_2$$



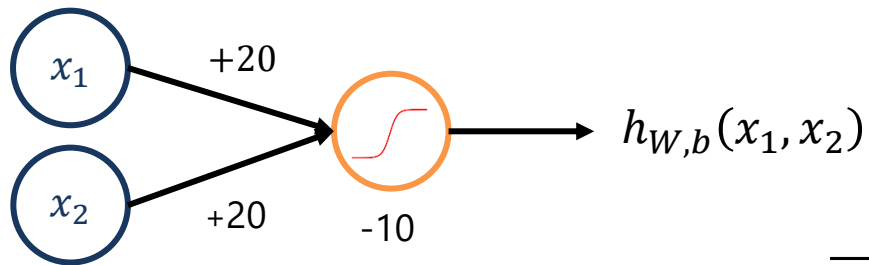
$$h_{\theta,b}(x_1, x_2) = g(-30 + 20x_1 + 20x_2)$$



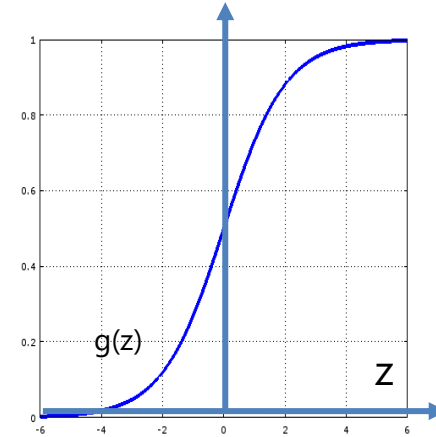
x_1	x_2	$h_{W,b}(x_1, x_2)$
0	0	$g(-30) \approx 0$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(10) \approx 1$

Example: OR

$$x_1, x_2 \in \{0, 1\}$$

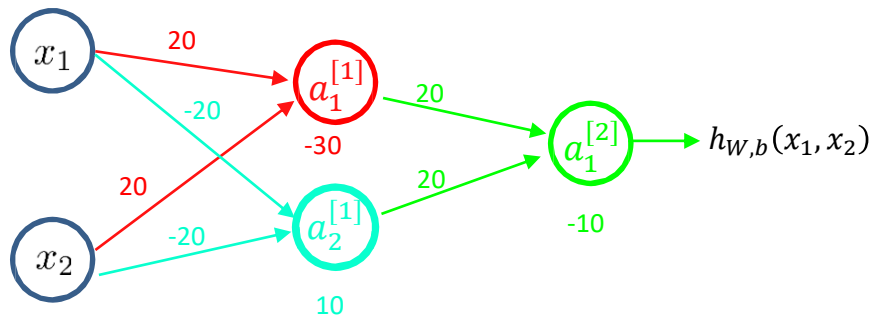
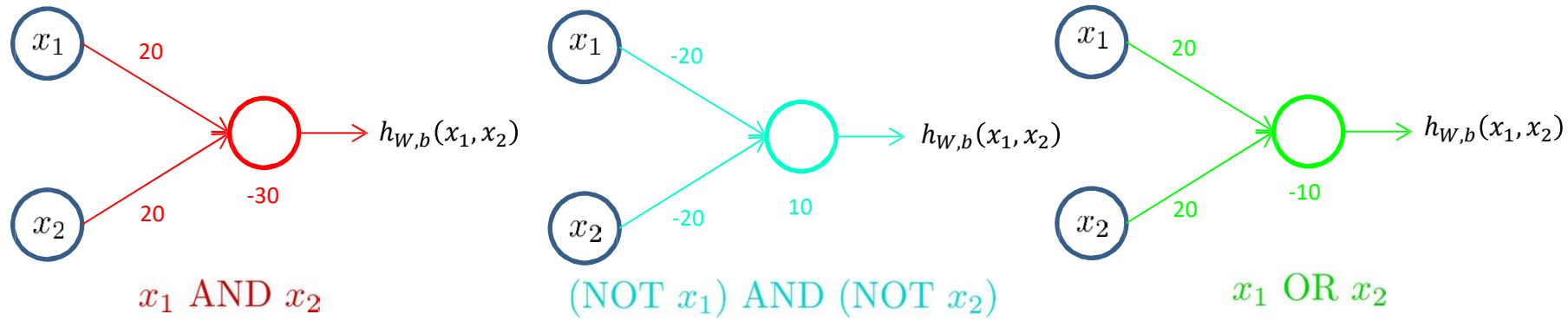


$$h_{\theta,b}(x_1, x_2) = g(-10 + 20x_1 + 20x_2)$$



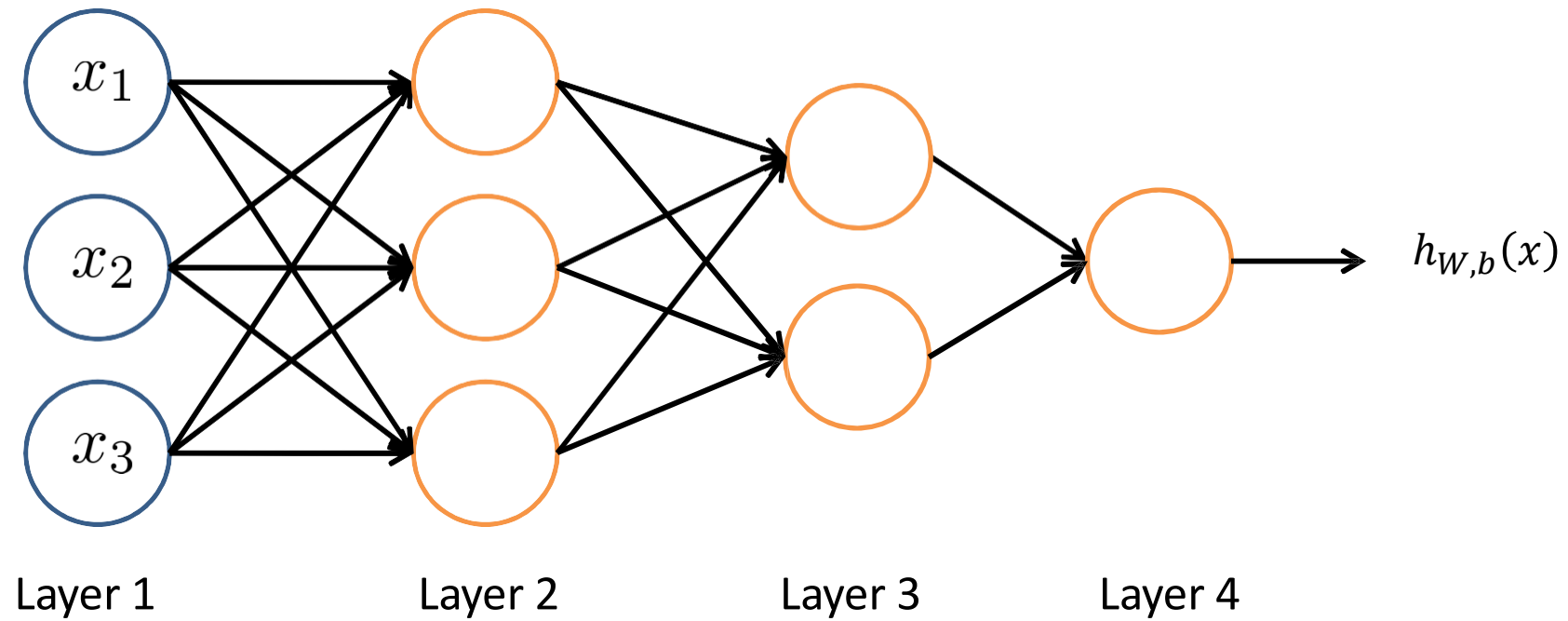
x_1	x_2	$h_{W,b}(x_1, x_2)$
0	0	$g(-10) \approx 0$
0	1	$g(10) \approx 1$
1	0	$g(10) \approx 1$
1	1	$g(30) \approx 1$

Example: XNOR



x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_{W,b}(x_1, x_2)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

Other network architectures

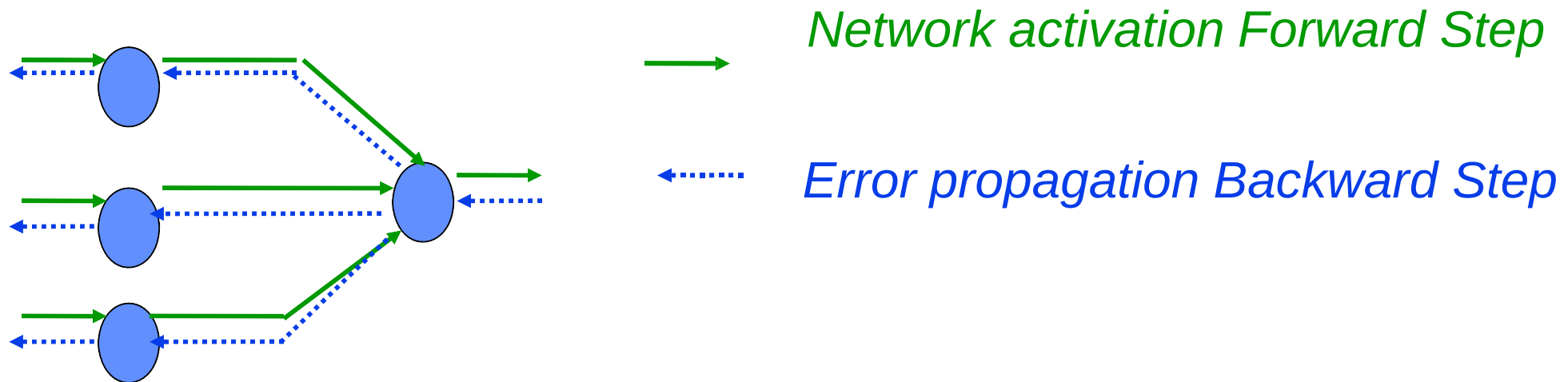


Training a Neural Network (computing weights)

Random initialization of the weights

Backpropagation consists of the repeated application of the following two steps:

- Forward step: in this step the network is activated on one example and the error of (each neuron of) the output layer is computed.
- Backward step: in this step the network error is used for updating the weights. Starting at the output layer, the error is propagated backwards through the network, layer by layer. This is done by recursively computing the local gradient of each neuron.



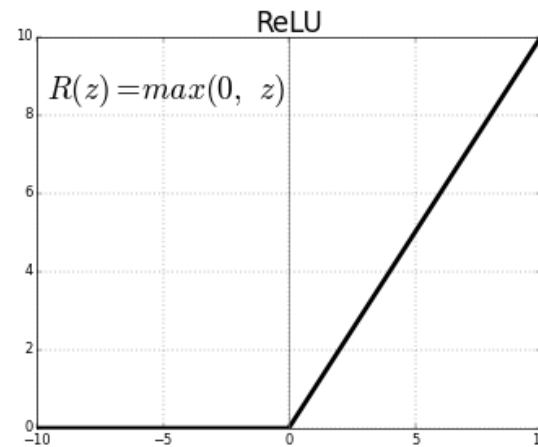
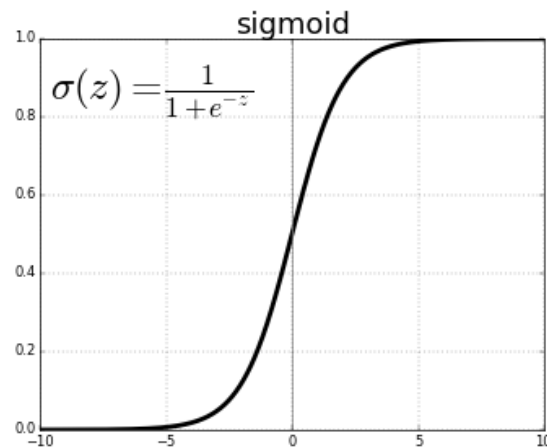
Activation functions

Sigmoid Function:

- it has seen frequent use historically, yet now it is rarely used, because saturates and kill the gradient.

Rectified Linear Unit (ReLu) Function:

- ReLu activation was found to greatly accelerate the convergence of SGD compared to Sigmoid function
- The implementation is simple w.r.t. other activations which require complex operations.



Loss Functions

For Binary Classification we often use the Binary Cross-Entropy Loss:

$$L(W, b) = \frac{1}{m} \sum_{i=1}^m -y^{(i)} \log(h_{W,b}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{W,b}(x^{(i)}))$$

Where m is the number of examples in the training set (see Cost Function of the Logistic Regression)

For Regression we often use the Mean Squared Error Loss:

$$L(W, b) = \frac{1}{m} \sum_{i=1}^m (h_{W,b}(x^{(i)}) - y^{(i)})^2$$

Where m is the number of examples in the training set (see Cost Function of the Linear Regression)

Note: in the regression setting the network ends with a single unit and no activation. Thus, the last layer is purely linear and the network is free to learn to predict values in any range.

Multiclass Classification



Pedestrian



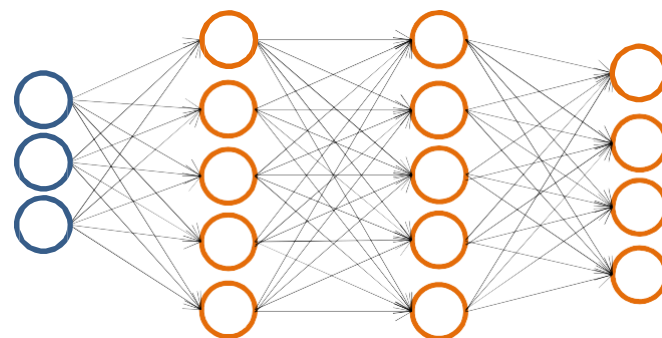
Car



Motorcycle



Truck



Target Labels: One-hot representation



Pedestrian



Car



Motorcycle



Truck

One-hot Representations:

$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Softmax Layer

In multiclass classification the last layer of a neural network often use a Softmax activation.

It means the network will output a *probability distribution* over the classes

Formally, let's $z^{[i]} = W^{[i]}a^{[i-1]} + b^{[i]}$

Softmax layer is:

$$a^{[i]} = \frac{e^{z^{[i]}}}{\sum_{j=1}^N e^{z_j^{[i]}}} \quad \text{where } N \text{ is the numbe of classes}$$

Example:

$$z^{[i]} = \begin{bmatrix} 5 \\ 2 \\ -1 \\ 3 \end{bmatrix} \rightarrow a^{[i]} = \begin{bmatrix} \frac{e^5}{t} \\ \frac{e^2}{t} \\ \frac{e^{-1}}{t} \\ \frac{e^3}{t} \end{bmatrix} \quad \text{where } t = (e^5 + e^2 + e^{-1} + e^3) \rightarrow a^{[i]} = \begin{bmatrix} 0.842 \\ 0.042 \\ 0.002 \\ 0.114 \end{bmatrix} \quad \left. \vphantom{\begin{bmatrix} 0.842 \\ 0.042 \\ 0.002 \\ 0.114 \end{bmatrix}} \right\} \text{The Sum is equal to 1}$$

Cross-Entropy Loss

To train a neural network for a multiclass problem we can use the Cross-Entropy Loss

Cross-entropy is commonly used to quantify the difference between two probability distributions.

Usually the "true" distribution (the one that your machine learning algorithm is trying to predict) is expressed in terms of a one-hot distribution.

$$L(W, b) = - \sum_{i=1}^m y^{(i)} \log(h_{W,b}(x^{(i)}))$$

Where m is the number of training samples, $y^{(i)}$ is the ground truth output distribution and $h_{W,b}(x^{(i)})$ is the predicted output distribution.

In practice, $y^{(i)}$ is a one-hot representation of the right label.

Cross-Entropy Loss

For Example:

$$y^{(i)} = [0 \ 1 \ 0]^T; h_{W,b}(x^{(i)}) = [0.228 \ 0.619 \ 0.153]^T$$

$$L = -(0 * \ln(0.228) + 1 * \ln(0.619) + 0 * \ln(0.153)) = 0.479$$

The **Cross-Entropy Loss** focuses on the **positive class**, because **the negative classes are potentially infinite**.

Another Example:

$$\text{Exactly right: } L = -(1 * \ln(1)) = 0$$

$$50\% \text{ Probability on correct target: } L = -(1 * \ln(0.5)) = 0.693$$

$$25\% \text{ Probability on correct target: } L = -(1 * \ln(0.25)) = 1.386$$

$$0\% \text{ Probability on correct target: } L = -(1 * \ln(0)) = \infty$$

